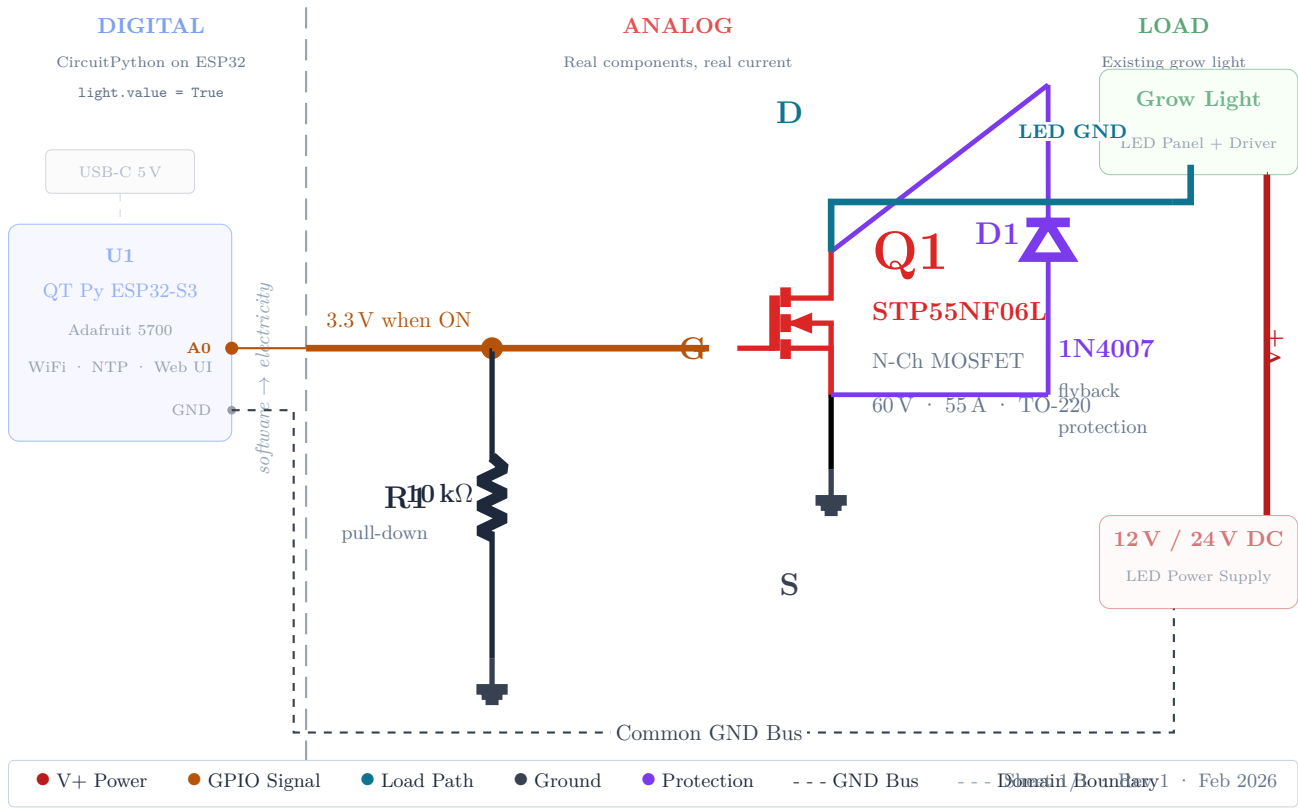


Herb Garden Light Controller

ESP32-S3 + MOSFET · CircuitPython · WiFi Scheduler

Schematic — Low-Side N-Channel MOSFET Switch



How It Works

- U1** (digital) connects to WiFi, syncs clock via NTP.
 - Schedule says ON → CircuitPython sets **A0** = **True**.
 - Pin **A0** outputs 3.3V. *Software becomes electricity*.
 - 3.3V on **Q1** gate (analog) opens the MOSFET channel.
 - Current: Grow Light → **Q1** D→S → GND. Light on.
 - Schedule says OFF → **A0** LOW → **Q1** closes. Light off.
- R1:** Keeps gate LOW during ESP32 boot (prevents light flash).
D1: Clamps voltage spikes when **Q1** switches off.

Bill of Materials — DigiKey

#	Ref	Component	DigiKey #	Qty	Price
1	U1	QT Py ESP32-S3	1528-5700-ND	1	\$12.50
2	Q1	STP55NF06L	497-6742-5-ND	2	\$2.07
3	D1	1N4007 Diode	1N4007DICT-ND	1	\$0.10
4	R1	10 kΩ 1/4W	CF14JT10K0CT-ND	5	\$0.10
5	–	Jumper wires	Global Spec.	1	\$5.60
6	–	Breadboard	(search)	1	~\$4

Total: ~\$28 · Order: [digikey.com/en/mylists](https://www.digikey.com/en/mylists)

Before starting: Open grow light base, find DC voltage (12V or 24V).

Software — CircuitPython

The ESP32 runs **CircuitPython**—Python that executes directly on the microcontroller. No compiler. No IDE. Plug in via USB-C, a flash drive appears, drag files onto it. Edit code, save, it runs.

Why this is cool

The ESP32 runs a tiny web server. Open a browser on your phone, visit <http://herbgarden.local>, and you get a scheduling UI. No app. No cloud. All running on a chip the size of a postage stamp under the kitchen cabinet.

File Structure on the ESP32

/code.py	Main program—runs on boot
/lib/	
adafruit_httpserver/	Web server library
adafruit_ntp.mpy	Network time sync
/static/	
index.html	Schedule UI (what your phone sees)
/schedule.json	Saved on/off times
/settings.toml	WiFi creds, timezone, quiet hours

The Main Loop (Simplified)

```
import time, board, digitalio, json, wifi, socketpool
import adafruit_ntp
from adafruit_httpserver import Server, Request, Response

# GPIO pin that controls the MOSFET gate
light = digitalio.DigitalInOut(board.A0)
light.direction = digitalio.Direction.OUTPUT

# Connect to WiFi (creds from settings.toml)
pool = socketpool.SocketPool(wifi.radio)
ntp = adafruit_ntp.NTP(pool, tz_offset=-8) # Pacific Time

# Load saved schedule
with open("/schedule.json", "r") as f:
    schedule = json.load(f)

server = Server(pool)

@server.route("/")
def index(request: Request):
    with open("/static/index.html", "r") as f:
        return Response(request, f.read(), content_type="text/html")

server.start(str(wifi.radio.ipv4_address))

while True:
    server.poll() # handle web requests
    now = ntp.datetime
    light.value = should_be_on(now, schedule) # True/False -> HIGH/LOW
    time.sleep(1)
```

Build Phases

Phase 1 — Prove it works (1 afternoon) Flash CircuitPython. Connect WiFi. Sync time. Toggle onboard LED on schedule. Web UI on phone. No MOSFET yet.

Phase 2 — Real load (1 evening) Wire MOSFET on breadboard. Connect to grow light's DC ground. Control actual light from web UI.

Phase 3 — Install (weekend) Mount under cabinet. Solder permanent connections. Set schedule. Forget about it.

Phase 4 — Show off (optional) GitHub. Blog post. MQTT for Home Assistant. Soil moisture sensor. Go wild.

First step when parts arrive

Plug QT Py into USB-C. A drive called CIRCUITPY appears. If it doesn't, visit circuitpython.org/downloads and flash the latest build for "Adafruit QT Py ESP32-S3." Then create `settings.toml` with your WiFi credentials.